

به نام خدا

درس سیستم های عامل

استاد: دکتر اسدی

پروژه چهارم: پیاده سازی سیستم Build در داکر

خلاصه پروژه

سارا قضاوی، زهرا قصابی، آرش قوامی، سید معین حسینی

402105908 – 402106359 – 402106337 – 402106348

پروژه چهارم: پیاده‌سازی سیستم Build در داکر

• مقدمه

هدف این پروژه آشنایی عملی با نحوه ساخته شدن ایمیج‌ها در داکر و درک بهتر سازوکار داخلی دستور build است. در این پروژه قرار است ابزار zocker پیاده‌سازی شود که به صورت ساده شده، فرایند build ایمیج‌های کانتینری را شبیه‌سازی می‌کند. تمرکز اصلی پروژه روی مفاهیم لایه‌های ایمیج، OverlayFS و مکانیزم کش در زمان build است. به صورت خلاصه می‌توان گفت به جای استفاده مستقیم از داکر، می‌خواهیم بفهمیم چه مرحله‌ای را طی می‌کند و این مراحل چگونه باعث سریع‌تر شدن build ایمیج‌ها می‌شوند.

• ایده کلی پروژه

در داکر هر ایمیج از چند لایه تشکیل شده است و هر دستور در Dockerfile معمولاً یک لایه جدید می‌سازد. داکر این لایه‌ها را نگه می‌دارد تا اگر در build های بعدی دوباره به آنها نیاز شد، به جای ساخت مجدد، از نسخه قبلی استفاده کند که این کار باعث افزایش سرعت build می‌شود. در این پروژه، همین منطق را در ابزار zocker پیاده‌سازی می‌کنیم، یعنی ایمیج‌ها به صورت مرحله مرحله ساخته می‌شوند، لایه‌ها ذخیره می‌شوند و در صورت امکان از کش استفاده می‌شود.

• بخش اول: ساخت ایمیج با Zockerfile

در بخش اول باید بتوان یک ایمیج را بر اساس فایل Zockerfile ساخت. این فایل ساختاری مانند Dockerfile دارد و شامل دستوراتی است که نحوه build ایمیج را مشخص می‌کنند. برخی از دستورات مهم عبارتند از:

دستور FROM: برای مشخص کردن ایمیج پایه

دستور RUN: برای اجرای یک دستور در محیط ایمیج

دستور COPY و ADD: برای اضافه کردن فایل به ایمیج

دستور WORKDIR: برای تعیین مسیر کاری

دستور ARG: برای متغیرهای زمان build

دستور CMD: برای مشخص کردن دستور اجرای نهایی ایمیج

در این پروژه zocker طراحی شده باید این دستورات را به ترتیب اجرا کند و بعد از هر مرحله، یک لایه جدید بسازد. هر لایه روی لایه‌های قبلی قرار می‌گیرد و وضعیت فایل‌ها و تغییرات آن مرحله را نگه می‌دارد.

• بخش دوم: پیاده‌سازی کش

در بخش دوم تمرکز روی پیاده‌سازی مکانیزم کش است. هدف این بخش این است که اگر یک ایمیج قبلاً ساخته شده و دوباره همان مراحل تکرار شوند، build سریع‌تر انجام شود. برای این کار در هر مرحله build یک مقدار hash ساخته می‌شود که بر اساس دستور اجرا شده و لایه قبلی است. اگر قبلاً لایه‌ای با همان hash وجود داشته باشد، آن مرحله اجرا نمی‌شود و از کش استفاده می‌شود، بنابراین فقط مرحله‌ای که تغییر کرده‌اند دوباره ساخته می‌شوند. در نتیجه زمان build کاهش می‌یابد.

• قابلیت‌های مدیریتی zocker

علاوه بر ساخت ایميج zocker باید چند دستور مدیریتی هم داشته باشد تا کار با ایميج ها ساده تر شود:

نمایش لیست ایميج های ساخته شده

حذف ایميج ها

نمایش تاریخچه لایه های هر ایميج

حذف لایه هایی که دیگر استفاده نمی شوند

این قابلیت ها کمک می کنند ساختار لایه ها و وضعیت ایميج ها بهتر قابل بررسی باشد.

• بخش امتیازی Multi-Stage Build:

در بخش امتیازی مفهوم Multi-Stage Build مطرح می شود. در این حالت یک Zockerfile می تواند چند مرحله build داشته باشد و خروجی یک مرحله در مرحله بعد استفاده شود. این روش باعث می شود ایميج نهایی حجم کمتری داشته باشد و فقط فایل های مورد نیاز در آن باقی بمانند.

جمع بندی

در مجموع این پروژه یک تمرین عملی برای فهم بهتر نحوه کار داکر در زمان build است. با پیاده سازی zocker مفاهیمی مانند لایه ها، کش و ساخت مرحله ای ایميج ها به صورت ملموس تری درک می شوند.